

COMPTE RENDU

BI & Datawarehouse – Devoir ETL
3e année Cybersécurité – École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha
	JIDAL Ilyas
	KABORE Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2025/2026
Enseignant :	M. Riadsolh
Date :	11 mai 2026

Table des matières

Description du projet	2
1 Architecture du Data Warehouse	3
1.1 Schéma en étoile	3
1.2 Description des tables	3
2 Création des tables (DDL)	3
2.1 Recréation de la base	3
2.2 Table <code>dim_date</code>	4
2.3 Table <code>dim_product</code>	4
2.4 Table <code>dim_age_group</code>	4
2.5 Tables de faits	5
3 Procédures ETL	5
3.1 <code>etl_load_dim_date</code> – Génération du calendrier	6
3.2 <code>etl_load_dim_product</code> – Chargement des produits	6
3.3 <code>etl_load_dim_age_group</code> – Tranches d'âge	7
3.4 <code>etl_load_facts</code> – Chargement des faits	7
3.4.1 Source commune : table temporaire <code>#SalesKeyed</code>	7
3.4.2 Chargement de <code>fact_sales</code> via <code>MERGE</code>	7
3.4.3 Chargement de <code>fact_monthly_sales</code> via <code>MERGE</code>	8
4 Démonstration	9
4.1 Exécution du script	9
4.2 Sortie console et bilan des chargements	9
4.3 Vérification de la dimension date et de l'année fiscale	9
4.4 Exemples de données dans <code>fact_sales</code>	10
4.5 Exemples de données dans <code>fact_monthly_sales</code>	10
4.6 Requête d'analyse : top 5 produits par chiffre d'affaires	10
4.7 Idempotence du script	11
Conclusion	11

Description du projet

Ce devoir porte sur la conception et l'implémentation d'un processus ETL (*Extract, Transform, Load*) en T-SQL pur pour alimenter un *Data Warehouse* dédié à l'analyse des ventes d'AdventureWorks.

La source de données est la base transactionnelle **AdventureWorks2022** (SQL Server 2022). La destination est une nouvelle base appelée **TIA_ETL**, créée et peuplée intégralement par un unique script SQL.

Périmètre des données

Les ventes prises en compte sont uniquement celles réalisées **auprès de clients particuliers**, c'est-à-dire les lignes de `Sales.Customer` vérifiant :

```
1 PersonID IS NOT NULL    -- client individuel (pas un magasin)
2 AND StoreID IS NULL     -- pas un client B2B
```

La plage temporelle couverte est le **1^{er} janvier 2010** au **31 décembre 2020**. Les données AdventureWorks utilisées s'étendent de mai 2011 à juin 2014, ce qui est entièrement inclus dans cette fenêtre.

Convention de l'année fiscale

Conformément au sujet, l'année fiscale d'AdventureWorks **commence le premier lundi du mois de mars** de chaque année calendaire. L'hypothèse `SET DATEFIRST 1` est appliquée (lundi = 1^{er} jour de la semaine).

La formule utilisée pour calculer le décalage jusqu'au premier lundi de mars est :

$$\text{offset} = (8 - \text{DATEPART}(\text{WEEKDAY}, 1 \text{ mars})) \bmod 7$$

Par exemple, le 1^{er} mars 2012 était un jeudi (`WEEKDAY = 4`) :

$$\text{offset} = (8 - 4) \bmod 7 = 4 \implies \text{AF 2012 commence le 5 mars 2012}$$

1 Architecture du Data Warehouse

Le Data Warehouse TIA_ETL suit un **schéma en étoile** composé de deux tables de faits reliées à trois dimensions.

1.1 Schéma en étoile

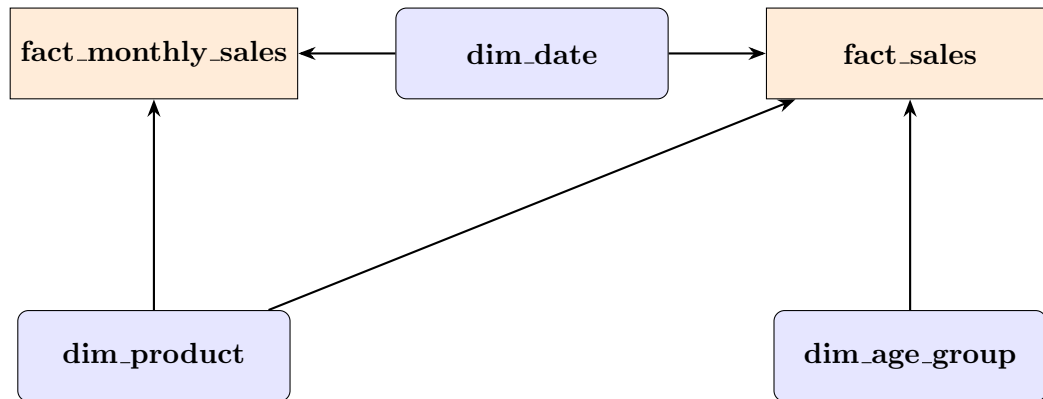


FIGURE 1 – Schéma en étoile de TIA_ETL

1.2 Description des tables

Table	Lignes	Granularité / Rôle
dim_date	4 018	Un enregistrement par jour (01/01/2010 – 31/12/2020)
dim_product	504	Un enregistrement par produit Adventure-Works
dim_age_group	4	Quatre tranches d'âge statiques
fact_sales	32 188	Ventes par (jour, produit, tranche d'âge)
fact_monthly_sales	1 941	Ventes agrégées par (mois, produit)

2 Création des tables (DDL)

Conformément aux consignes, la création des objets est réalisée **en dehors de toute procédure enregistrée**. Le script commence par (re)créer la base TIA_ETL, puis définit les cinq tables.

2.1 Recréation de la base

```
1 USE master;
2 IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'TIA_ETL')
3 BEGIN
4     ALTER DATABASE TIA_ETL SET SINGLE_USER WITH ROLLBACK
5     IMMEDIATE;
6     DROP DATABASE TIA_ETL;
7 END
8 CREATE DATABASE TIA_ETL;
9 USE TIA_ETL;
```

2.2 Table dim_date

```

1 CREATE TABLE dim_date (
2     date_id          INT          NOT NULL,      -- YYYYMMDD
3     date_value       DATETIME     NOT NULL,
4     month_num        TINYINT      NOT NULL,      -- 1..12
5     month_name       VARCHAR(20)  NOT NULL,
6     quarter_num      TINYINT      NOT NULL,      -- 1..4
7     quarter_name     CHAR(2)      NOT NULL,      -- 'Q1'..'Q4'
8     year_num         SMALLINT     NOT NULL,
9     month_year       VARCHAR(10)  NOT NULL,      -- ex. '2010-1'
10    quarter_year      VARCHAR(10)  NOT NULL,      -- ex. '2010-Q1'
11    fiscal_year       SMALLINT     NOT NULL,
12    fiscal_week       SMALLINT     NOT NULL,      -- 1..53
13    fiscal_year_week  VARCHAR(10)  NOT NULL,      -- ex. '2010-1'
14    CONSTRAINT PK_dim_date PRIMARY KEY (date_id)
15 );

```

Listing 2 – DDL de dim_date

2.3 Table dim_product

```

1 CREATE TABLE dim_product (
2     product_id       INT          NOT NULL,
3     product_name     NVARCHAR(50) NOT NULL,
4     product_number   NVARCHAR(25) NOT NULL,
5     color            NVARCHAR(15) NULL,
6     standard_cost    MONEY        NOT NULL,
7     list_price       MONEY        NOT NULL,
8     style            NVARCHAR(5)  NULL,
9     subcategory_name NVARCHAR(50) NULL,
10    category_name     NVARCHAR(50) NULL,
11    CONSTRAINT PK_dim_product PRIMARY KEY (product_id)
12 );

```

Listing 3 – DDL de dim_product

2.4 Table dim_age_group

```

1 CREATE TABLE dim_age_group (
2     age_group_id     INT          NOT NULL,
3     age_group_label  VARCHAR(30)  NOT NULL,
4     age_min          INT          NULL,          -- NULL = pas de borne
5     basse            basse
6     age_max          INT          NULL,          -- NULL = pas de borne
7     haute            haute

```

```

6      CONSTRAINT PK_dim_age_group PRIMARY KEY (age_group_id)
7  );

```

Listing 4 – DDL de dim_age_group

2.5 Tables de faits

```

1  -- Fait transactionnel : (jour, produit, tranche d'age)
2  CREATE TABLE fact_sales (
3      date_id          INT      NOT NULL,
4      product_id       INT      NOT NULL,
5      age_group_id     INT      NOT NULL,
6      total_quantity   INT      NOT NULL,
7      total_amount     MONEY    NOT NULL,
8      CONSTRAINT PK_fact_sales
9          PRIMARY KEY (date_id, product_id, age_group_id),
10     CONSTRAINT FK_fact_sales_date
11         FOREIGN KEY (date_id) REFERENCES dim_date(date_id),
12     CONSTRAINT FK_fact_sales_product
13         FOREIGN KEY (product_id) REFERENCES dim_product(
14             product_id),
15     CONSTRAINT FK_fact_sales_age
16         FOREIGN KEY (age_group_id) REFERENCES dim_age_group(
17             age_group_id)
18 );
19
20 -- Fait mensuel agrege : (mois, produit) -- alimente via MERGE
21 CREATE TABLE fact_monthly_sales (
22     date_id          INT      NOT NULL,    -- date_id du 1er jour du
23     mois
24     product_id       INT      NOT NULL,
25     total_quantity   INT      NOT NULL,
26     order_count      INT      NOT NULL,    -- nb de commandes
27     distinctes
28     total_amount     MONEY    NOT NULL,
29     CONSTRAINT PK_fact_monthly_sales
30         PRIMARY KEY (date_id, product_id),
31     CONSTRAINT FK_fact_monthly_date
32         FOREIGN KEY (date_id) REFERENCES dim_date(date_id),
33     CONSTRAINT FK_fact_monthly_product
34         FOREIGN KEY (product_id) REFERENCES dim_product(
35             product_id)
36 );

```

Listing 5 – DDL de fact_sales et fact_monthly_sales

3 Procédures ETL

Quatre procédures enregistrées implémentent la logique ETL. Elles sont appelées en séquence à la fin du script.

3.1 etl_load_dim_date – Génération du calendrier

Cette procédure génère les 4018 lignes de `dim_date` (01/01/2010 – 31/12/2020) à l'aide de deux CTE récursives.

CTE FiscalStarts Précalcule le premier lundi de mars pour les années 2009 à 2021 grâce à la formule d'offset.

CTE DateSeries Génère la série de dates par récurrence avec l'option `MAXRECURSION 5000`.

CTE DateFiscal Déduit pour chaque date son année fiscale et la date de début de cette année fiscale, ce qui permet de calculer la semaine fiscale par :

$$\text{fiscal_week} = \lfloor \text{DATEDIFF}(\text{DAY}, \text{fiscal_start}, d) / 7 \rfloor + 1$$

```
1 WITH FiscalStarts AS (  
2     SELECT y,  
3         DATEADD(DAY,  
4             (8 - DATEPART(WEEKDAY, DATEFROMPARTS(y, 3, 1))) %  
5             7,  
6             DATEFROMPARTS(y, 3, 1)) AS fs  
7     FROM (VALUES  
8         (2009), (2010), (2011), (2012), (2013), (2014), (2015),  
9         (2016), (2017), (2018), (2019), (2020), (2021)  
10    ) v(y)  
11 ),
```

Listing 6 – Calcul du début d'année fiscale

3.2 etl_load_dim_product – Chargement des produits

Importe l'intégralité du catalogue produit depuis `AdventureWorks2022` via des jointures `LEFT JOIN` sur les tables `ProductSubcategory` et `ProductCategory`.

```
1 INSERT INTO dim_product (  
2     product_id, product_name, product_number,  
3     color, standard_cost, list_price, style,  
4     subcategory_name, category_name  
5 )  
6 SELECT  
7     p.ProductID, p.Name, p.ProductNumber,  
8     p.Color, p.StandardCost, p.ListPrice, RTRIM(p.Style),  
9     ps.Name AS subcategory_name,  
10    pc.Name AS category_name  
11 FROM AdventureWorks2022.Production.Product p  
12 LEFT JOIN AdventureWorks2022.Production.ProductSubcategory ps  
13     ON p.ProductSubcategoryID = ps.ProductSubcategoryID  
14 LEFT JOIN AdventureWorks2022.Production.ProductCategory pc  
15     ON ps.ProductCategoryID = pc.ProductCategoryID;
```

Listing 7 – Extraction des produits depuis `AdventureWorks2022`

3.3 etl_load_dim_age_group – Tranches d’âge

Insère les quatre tranches d’âge statiques définies par le sujet :

ID	Libellé	Borne inf.	Borne sup.
1	-25 ans	–	24
2	25-34 ans	25	34
3	35-44 ans	35	44
4	45 ans et plus	45	–

3.4 etl_load_facts – Chargement des faits

3.4.1 Source commune : table temporaire #SalesKeyed

Pour alimenter les deux tables de faits à partir d’une source unique (comme recommandé par le sujet), les données sont d’abord matérialisées dans une table temporaire #SalesKeyed.

Les jointures réalisées sont :

- SalesOrderHeader ⋈ Customer (filtre clients particuliers)
- SalesOrderHeader ⋈ SalesOrderDetail
- CROSS APPLY sur Person.Person pour extraire la date de naissance depuis le champ XML Demographics

L’âge exact à la date de commande est calculé avec la formule suivante, qui corrige le décalage lorsque l’anniversaire n’est pas encore passé dans l’année :

```
1 DATEDIFF(YEAR, bd.BirthDate, soh.OrderDate)
2 - CASE
3     WHEN MONTH(bd.BirthDate) > MONTH(soh.OrderDate)
4         OR (MONTH(bd.BirthDate) = MONTH(soh.OrderDate)
5             AND DAY(bd.BirthDate) > DAY(soh.OrderDate))
6     THEN 1 ELSE 0
7 END AS age
```

Listing 8 – Calcul de l’âge exact

L’âge est ensuite converti en identifiant de tranche d’âge :

```
1 CASE
2     WHEN age < 25 THEN 1 -- '-25 ans'
3     WHEN age <= 34 THEN 2 -- '25-34 ans'
4     WHEN age <= 44 THEN 3 -- '35-44 ans'
5     ELSE 4 -- '45 ans et plus'
6 END AS age_group_id
```

Listing 9 – Mapping age -> age_group_id

3.4.2 Chargement de fact_sales via MERGE

La granularité de fact_sales est (jour × produit × tranche d’âge).


```

1 MERGE INTO fact_sales AS tgt
2 USING (
3     SELECT date_id, ProductID AS product_id, age_group_id,
4           SUM(OrderQty) AS total_quantity,
5           SUM(LineTotal) AS total_amount
6     FROM #SalesKeyed
7     GROUP BY date_id, ProductID, age_group_id
8 ) AS src
9 ON   tgt.date_id      = src.date_id
10 AND tgt.product_id   = src.product_id
11 AND tgt.age_group_id = src.age_group_id
12 WHEN MATCHED THEN
13     UPDATE SET total_quantity = src.total_quantity,
14               total_amount   = src.total_amount
15 WHEN NOT MATCHED THEN
16     INSERT (date_id, product_id, age_group_id,
17           total_quantity, total_amount)
18     VALUES (src.date_id, src.product_id, src.age_group_id,
19           src.total_quantity, src.total_amount);

```

Listing 10 – MERGE pour fact_sales

3.4.3 Chargement de fact_monthly_sales via MERGE

La granularité de fact_monthly_sales est (**mois × produit**). La clé de date utilisée est le date_id du **premier jour du mois** (format YYYYMM01).

L'utilisation de MERGE permet d'alimenter la table de manière incrémentale : si de nouvelles lignes arrivent alors qu'un mois n'est pas encore clôturé, la procédure met à jour les agrégats existants plutôt que de créer des doublons.

Le nombre de commandes est compté avec COUNT(DISTINCT SalesOrderID) pour ne compter que les **commandes** (et non les lignes de commandes).

```

1 MERGE INTO fact_monthly_sales AS tgt
2 USING (
3     SELECT month_date_id      AS date_id,
4           ProductID          AS product_id,
5           SUM(OrderQty)      AS total_quantity,
6           COUNT(DISTINCT SalesOrderID) AS order_count,
7           SUM(LineTotal)     AS total_amount
8     FROM #SalesKeyed
9     GROUP BY month_date_id, ProductID
10 ) AS src
11 ON   tgt.date_id      = src.date_id
12 AND tgt.product_id   = src.product_id
13 WHEN MATCHED THEN
14     UPDATE SET total_quantity = src.total_quantity,
15               order_count    = src.order_count,
16               total_amount   = src.total_amount
17 WHEN NOT MATCHED THEN
18     INSERT (date_id, product_id, total_quantity,
19           order_count, total_amount)

```

```

20 VALUES (src.date_id, src.product_id,
21          src.total_quantity, src.order_count, src.total_amount
          );

```

Listing 11 – MERGE pour fact_monthly_sales

4 Démonstration

4.1 Exécution du script

Le script est exécuté depuis un terminal Linux en une seule commande, après copie dans le conteneur Docker exécutant SQL Server 2022 :

```

docker cp etl_tia.sql sqlserver:/tmp/etl_tia.sql
docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
  -S localhost -U sa -P 'Admin@1234' -No -i /tmp/etl_tia.sql

```

4.2 Sortie console et bilan des chargements

```

Changed database context to 'master'.
Changed database context to 'TIA_ETL'.
>>> Base TIA_ETL creee.
>>> Tables creees.
>>> Procedures ETL creees.
--- Chargement dim_date ...
    dim_date : 4018 lignes chargees.
--- Chargement dim_product ...
    dim_product : 504 lignes chargees.
--- Chargement dim_age_group ...
    dim_age_group : 4 lignes chargees.
--- Chargement facts ...
    fact_sales charge.
    fact_monthly_sales charge.

=== BILAN DES CHARGEMENTS ===
Table          Lignes
-----
dim_date              4018
dim_product           504
dim_age_group         4
fact_sales            32188
fact_monthly_sales    1941
>>> ETL termine avec succes.

```

4.3 Vérification de la dimension date et de l'année fiscale

La sortie suivante démontre la justesse du calcul de l'année fiscale : les 4 premiers jours de mars 2012 appartiennent encore à l'AF 2011 (semaine 52), tandis que le 5 mars 2012 – premier lundi de mars – marque le début de l'AF 2012 (semaine 1).

```

--- Exemple dim_date (fiscal year 2012) ---
date_id  date_value      month_name quarter year fiscal_year fweek fyw
-----
-----

```

20120301	2012-03-01	...	March	Q1	2012	2011	52
	2011-52						
20120302	2012-03-02	...	March	Q1	2012	2011	52
	2011-52						
20120303	2012-03-03	...	March	Q1	2012	2011	52
	2011-52						
20120304	2012-03-04	...	March	Q1	2012	2011	52
	2011-52						
20120305	2012-03-05	...	March	Q1	2012	2012	1
	2012-1						

Note : De même, l'AF 2011 a commencé le 7 mars 2011, tel que spécifié dans le sujet, ce qui est bien vérifié en base :

20110305	2011-03-05	...	March	Q1	2011	2010	53	2010-53
20110306	2011-03-06	...	March	Q1	2011	2010	53	2010-53
20110307	2011-03-07	...	March	Q1	2011	2011	1	2011-1

4.4 Exemples de données dans fact_sales

```
--- Exemple fact_sales ---
```

date_id	date_value		product_name	age_group	qty
amount					
20110531	2011-05-31	...	Road-150 Red, 62	45 ans et +	1
3578.27					
20110531	2011-05-31	...	Road-650 Black, 62	45 ans et +	1
699.10					
20110531	2011-05-31	...	Mountain-100 Silver, 44	35-44 ans	1
3399.99					
20110531	2011-05-31	...	Mountain-100 Silver, 44	45 ans et +	2
6799.98					
20110601	2011-06-01	...	Road-150 Red, 62	35-44 ans	1
3578.27					

4.5 Exemples de données dans fact_monthly_sales

```
--- Exemple fact_monthly_sales ---
```

date_id	month_year	product_name	qty	orders	amount
20110501	2011-5	Road-150 Red, 62	1	1	3578.27
20110501	2011-5	Road-650 Black, 62	1	1	699.10
20110501	2011-5	Mountain-100 Silver, 44	3	3	10199.97
20110601	2011-6	Road-150 Red, 62	21	21	75143.67
20110601	2011-6	Road-150 Red, 44	23	23	82300.21

4.6 Requête d'analyse : top 5 produits par chiffre d'affaires

```

1 SELECT TOP 5
2     dp.product_name,
3     SUM(fs.total_amount) AS ca
4 FROM fact_sales fs
5 JOIN dim_product dp ON fs.product_id = dp.product_id

```

```

6 GROUP BY dp.product_name
7 ORDER BY ca DESC;

```

Listing 12 – Top 5 produits par CA

```

product_name          ca
-----
Mountain-200 Black, 46    1373469.5482
Mountain-200 Black, 42    1363142.0934
Mountain-200 Silver, 38   1339462.7904
Mountain-200 Silver, 46   1301100.0984
Mountain-200 Black, 38    1294866.1412

(5 rows affected)

```

4.7 Idempotence du script

Le script a été exécuté plusieurs fois consécutivement : les comptages restent identiques à chaque exécution (4 018, 504, 4, 32 188, 1 941 lignes respectivement), ce qui confirme l'idempotence du processus ETL.

Conclusion

Ce devoir a permis de concevoir et d'implémenter un processus ETL complet en T-SQL pour alimenter le Data Warehouse TIA.ETL à partir de la base transactionnelle AdventureWorks2022. Les étapes couvertes sont :

- **Extraction** : jointure de six tables (`SalesOrderHeader`, `Customer`, `SalesOrderDetail`, `Person`, `Product`, `ProductSubcategory/ProductCategory`) avec filtre sur les clients particuliers (`PersonID IS NOT NULL`).
- **Transformation** :
 - Génération du calendrier 2010–2020 avec attributs calendaires et fiscaux (CTE récursive).
 - Calcul de l'année fiscale et de la semaine fiscale via la formule du premier lundi de mars.
 - Calcul de l'âge exact du client à la date de commande à partir du champ XML `Demographics`.
 - Affectation de la tranche d'âge correspondante.
- **Chargement** :
 - DDL hors procédures enregistrées, conforme aux consignes.
 - Utilisation de `MERGE` pour les deux tables de faits, garantissant la cohérence lors de chargements successifs (notamment en cours de mois pour `fact_monthly_sales`).
 - Table temporaire `#SalesKeyed` partagée entre les deux `MERGE` : la source AdventureWorks n'est interrogée qu'une seule fois.
 - 32 188 lignes dans `fact_sales` et 1 941 lignes dans `fact_monthly_sales` chargées avec succès.